Bhai, badhai ho! Part 1 complete kar liya, rule-based wala solid chal raha hai. Ab Part 2 mein hum ise **AI-powered** banayenge — NLP use karke intent samjhega (jaise user "fees kitni hai?" bole to "fees" intent pakdega), better matching karega, synonyms/typos handle karega thoda, aur accurate response dega.

Hum **NLTK** (tokenize, lemmatize ke liye) + **scikit-learn** (TF-IDF + cosine similarity ke liye) use karenge. Yeh simple hai, beginner-friendly, aur doc mein NLTK mentioned hai. Deep learning (TensorFlow/PyTorch) optional hai, hum abhi skip karte hain kyuki beginner ho, baad mein add kar sakte hain.

Tech Updates for Part 2:

- Add libraries: NLTK, scikit-learn, numpy

- Continue Flask + MySQL

- Same folder: rule_based_chatbot (ab AI version)

- Training data: Intents (tag + response) + multiple patterns per intent

- Matching: User message ko preprocess → TF-IDF vector → cosine similarity se best intent find → response

- Threshold: 0.4 se kam similarity to fallback

- Admin panel upgrade: Intents manage + patterns add/edit/delete

- Chat logs same rahega

- No Jupyter needed abhi (simple code hai, VS Code mein hi chalega)

Part 2 Roadmap (Total 6 Phases)

1. **Phase 1: Additional Libraries & NLTK Setup**

2. **Phase 2: Database Update – Intents & Patterns Tables**

3. **Phase 3: Load Intents from DB & Build NLP Model**

4. **Phase 4: Upgrade /chat Route with AI Logic**

5. **Phase 5: Update Admin Panel for Intents & Patterns**

6. **Phase 6: Testing, Polish & Final Recap**

Chalo shuru karte hain. Har phase ke baad checklist.

Phase 1: Additional Libraries & NLTK Setup

Goal: New packages install karo, NLTK data download.

What we build: App mein imports add + NLTK ready.

Commands (venv activated hona chahiye):
```
pip install nltk scikit-learn numpy
pip freeze > requirements.txt
```

app.py mein top pe add karo (imports ke paas):
```python
import nltk
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# Download NLTK data (sirf pehli baar)
nltk.download('punkt', quiet=True)
nltk.download('wordnet', quiet=True)
```

Run karo:

`python app.py`

Koi error nahi aana chahiye.

Common mistakes:

- Download fail? → Internet check, ya manual download from nltk.org.

- Import error? → pip install sahi se.

Next Phase Checklist:

☑ New packages installed?

☑ app.py run without error?

Phase 2: Database Update – Intents & Patterns Tables

Goal: Old rules ko replace karo better structure se (intents + multiple patterns).

New Schema (old rules table drop kar sakte ho ya ignore):

```sql
-- New tables
CREATE TABLE intents (
    id INT AUTO_INCREMENT PRIMARY KEY,
    tag VARCHAR(100) NOT NULL,
    response TEXT NOT NULL
);

CREATE TABLE patterns (
    id INT AUTO_INCREMENT PRIMARY KEY,
    intent_id INT NOT NULL,
    pattern VARCHAR(255) NOT NULL,
```

```
    FOREIGN KEY (intent_id) REFERENCES intents(id) ON DELETE CASCADE
);

-- Optional: fallback intent (always last)
INSERT INTO intents (tag, response) VALUES
('greeting', 'Hello! Kaise help kar sakta hoon? 😊 '),
('goodbye', 'Bye! Take care aur jaldi milte hain! 👋 '),
('fees', 'Our fees are 5000 per semester. Kuch aur puchna hai?'),
('appointment', 'Appointment book karne ke liye call 1234567890 karo ya "book" bolo.'),
('how_are_you', 'Main theek hoon! Tu bata kaise hai?'),
('name', 'Mera naam AI Bot hai! Tera naam kya hai?'),
('fallback', 'Sorry bhai, samajh nahi aaya. Thoda clear bolo ya try karo: hi, fees, appointment, bye');
```

Sample Patterns (MySQL mein run karo, intent id note karo):

```sql
-- Assume greeting id=1, goodbye=2, fees=3, etc. (SELECT * FROM intents se id dekho)
INSERT INTO patterns (intent_id, pattern) VALUES
(1, 'hi'), (1, 'hello'), (1, 'hey'), (1, 'good morning'), (1, 'namaste'),
(2, 'bye'), (2, 'goodbye'), (2, 'see you'), (2, 'tata'),
(3, 'fees'), (3, 'fee'), (3, 'kitni fees'), (3, 'cost'), (3, 'paise'),
(4, 'appointment'), (4, 'book appointment'), (4, 'slot'), (4, 'time book'),
(5, 'how are you'), (5, 'kaisa hai'), (5, 'kaise ho'),
(6, 'what is your name'), (6, 'naam kya hai'), (6, 'who are you'),
(7, 'default');  -- fallback ke liye ek dummy pattern
```

Common mistakes:

- Foreign key error? → intents table pehle populate karo.

- Id galat? → SELECT id, tag FROM intents; se check.

Next Phase Checklist:

☑ New tables bana?

☑ intents mein 7 rows?

☑ patterns mein multiple entries per intent?

Phase 3: Load Intents & Build NLP Model

Goal: DB se data load karo, TF-IDF model banao.

Code (app.py mein add karo, imports ke neeche):

```python
# Global variables for NLP
intents = []
flat_patterns = []
vectorizer = None
X = None
lemmatizer = WordNetLemmatizer()

def clean_text(text):
    text = text.lower()
    tokens = nltk.word_tokenize(text)
    tokens = [lemmatizer.lemmatize(word) for word in tokens if word.isalnum()]
    return ' '.join(tokens)
```

```
def reload_nlp_model():
    global intents, flat_patterns, vectorizer, X

    conn = get_db_connection()
    cursor = conn.cursor(dictionary=True)

    # Load intents
    cursor.execute("SELECT * FROM intents")
    intents_data = cursor.fetchall()
    intents_dict = {row['id']: {'tag': row['tag'], 'response': row['response'], 'patterns': []} for row
in intents_data}

    # Load patterns
    cursor.execute("SELECT * FROM patterns")
    for row in cursor.fetchall():
        if row['intent_id'] in intents_dict:
            intents_dict[row['intent_id']]['patterns'].append(row['pattern'])

    intents = list(intents_dict.values())

    # Build flat list for TF-IDF
    flat_patterns = []
    for intent in intents:
        for pattern in intent['patterns']:
            flat_patterns.append({'intent': intent, 'pattern': pattern})

    if not flat_patterns:
        print("Warning: No patterns found!")
```

```
        return

    # Preprocess corpus
    corpus = [clean_text(fp['pattern']) for fp in flat_patterns]

    vectorizer = TfidfVectorizer()
    vectorizer.fit(corpus)
    X = vectorizer.transform(corpus)

    cursor.close()
    conn.close()
    print("NLP Model reloaded successfully!")

# Call once at start
reload_nlp_model()
```

Add reload_nlp_model() call in admin routes (POST, PUT, DELETE ke finally mein):

```python
reload_nlp_model()
```

Test: Run app.py, terminal mein "NLP Model reloaded" dikhega.

Common mistakes:

- No patterns? → DB mein data daalo.
- Vectorizer error? → corpus empty nahi hona chahiye.

Next Phase Checklist:

☑ reload_nlp_model() chal raha?

☑ Terminal mein success message?

Phase 4: Upgrade /chat Route with AI Logic

Goal: Old rule-based ko replace karo NLP se.

Replace purana /chat route (poora replace kar do):

```python
@app.route('/chat', methods=['POST'])
def chat():
    data = request.json
    user_message = data.get('message', '').strip()
    if not user_message:
        return jsonify({"response": "Kuch type karo na!"})

    # Preprocess user input
    user_clean = clean_text(user_message)
    if not user_clean.strip():
        return jsonify({"response": "Sorry, samajh nahi aaya."})

    user_vec = vectorizer.transform([user_clean])
    sims = cosine_similarity(user_vec, X)[0]
    max_sim = np.max(sims)

    if max_sim > 0.4:  # Adjust threshold if needed
        idx = np.argmax(sims)
```

```
        matched_intent = flat_patterns[idx]['intent']
        response = matched_intent['response']
    else:
        # Fallback
        fallback = next((i for i in intents if i['tag'] == 'fallback'), None)
        response = fallback['response'] if fallback else "Sorry, samajh nahi aaya. Try: hi, fees, bye"

    # Log to DB
    try:
        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.execute(
            "INSERT INTO chat_logs (user_message, bot_response) VALUES (%s, %s)",
            (user_message, response)
        )
        conn.commit()
    except Error as e:
        print("Log error:", e)
    finally:
        cursor.close()
        conn.close()

    return jsonify({"response": response})
```

Test (Postman ya UI):

- "hi there" → greeting response

- "fees kya hai" → fees response (even if not exact)

- "random xyz" → fallback

Common mistakes:

- Similarity low? → Threshold 0.3 kar do test ke liye.

- Import np error? → import numpy as np add karo.

Next Phase Checklist:

☑ "hello" se greeting?

☑ "fee structure bata" se fees?

☑ Logs DB mein?

Phase 5: Update Admin Panel for Intents & Patterns

Goal: Admin ko intents + patterns manage karne do.

New Routes (app.py mein add, old /admin/rules replace ya comment kar do):

```python
@app.route('/admin/intents', methods=['GET'])
def get_intents():
    conn = get_db_connection()
    cursor = conn.cursor(dictionary=True)
    cursor.execute("SELECT * FROM intents")
    data = cursor.fetchall()
    cursor.close()
    conn.close()
    return jsonify(data)
```

```
@app.route('/admin/intents', methods=['POST'])
def add_intent():
    data = request.json
    tag = data.get('tag')
    response = data.get('response')
    if not tag or not response:
        return jsonify({"error": "Tag and response required"}), 400
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute("INSERT INTO intents (tag, response) VALUES (%s, %s)", (tag, response))
    conn.commit()
    new_id = cursor.lastrowid
    cursor.close()
    conn.close()
    reload_nlp_model()
    return jsonify({"message": "Intent added", "id": new_id}), 201

@app.route('/admin/intents/<int:intent_id>', methods=['PUT'])
def update_intent(intent_id):
    data = request.json
    tag = data.get('tag')
    response = data.get('response')
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute("UPDATE intents SET tag=%s, response=%s WHERE id=%s", (tag, response,
intent_id))
    conn.commit()
    cursor.close()
```

```
    conn.close()
    reload_nlp_model()
    return jsonify({"message": "Intent updated"})

@app.route('/admin/intents/<int:intent_id>', methods=['DELETE'])
def delete_intent(intent_id):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute("DELETE FROM intents WHERE id=%s", (intent_id,))
    conn.commit()
    cursor.close()
    conn.close()
    reload_nlp_model()
    return jsonify({"message": "Intent deleted"})

# Patterns routes
@app.route('/admin/patterns', methods=['POST'])
def add_pattern():
    data = request.json
    intent_id = data.get('intent_id')
    pattern = data.get('pattern')
    if not intent_id or not pattern:
        return jsonify({"error": "intent_id and pattern required"}), 400
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute("INSERT INTO patterns (intent_id, pattern) VALUES (%s, %s)", (intent_id,
pattern))
    conn.commit()
```

```
    cursor.close()
    conn.close()
    reload_nlp_model()
    return jsonify({"message": "Pattern added"}), 201

@app.route('/admin/patterns/<int:pattern_id>', methods=['DELETE'])
def delete_pattern(pattern_id):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute("DELETE FROM patterns WHERE id=%s", (pattern_id,))
    conn.commit()
    cursor.close()
    conn.close()
    reload_nlp_model()
    return jsonify({"message": "Pattern deleted"})
```

Update admin.html (templates/admin.html poora replace ya update karo):

- Add sections for Intents & Patterns.
- Use JS to load intents, add new intent, add pattern (dropdown for intent).

Yeh thoda complex hai, isliye simple version:

```html
<!-- Inside <div class="chat-container"> after h2 -->
<h3>Manage Intents</h3>
<input id="intent_tag" placeholder="Tag (e.g. greeting)" style="width:100%; padding:8px;">
<textarea id="intent_response" placeholder="Response" style="width:100%; height:60px;"></textarea>
```

```
<button onclick="addIntent()">Add Intent</button>

<table id="intents-table">
  <thead><tr><th>ID</th><th>Tag</th><th>Response</th><th>Actions</th></tr></thead>
  <tbody></tbody>
</table>

<h3>Add Pattern to Intent</h3>
<select id="intent_select" style="width:100%; padding:8px; margin-bottom:10px;"></select>
<input id="new_pattern" placeholder="New pattern (e.g. namaste)">
<button onclick="addPattern()">Add Pattern</button>

<!-- JS update -->
<script>
function loadIntents() {
  fetch('/admin/intents')
    .then(res => res.json())
    .then(intents => {
      const tbody = document.querySelector('#intents-table tbody');
      tbody.innerHTML = '';
      const select = document.getElementById('intent_select');
      select.innerHTML = '';
      intents.forEach(intent => {
        // Table row
        const tr = document.createElement('tr');
        tr.innerHTML = `
          <td>${intent.id}</td>
          <td contenteditable="true" id="tag-${intent.id}">${intent.tag}</td>
```

```
            <td contenteditable="true" id="resp-${intent.id}">${intent.response}</td>
            <td>
              <button onclick="updateIntent(${intent.id})">Update</button>
              <button onclick="deleteIntent(${intent.id})">Delete</button>
            </td>`;
          tbody.appendChild(tr);

          // Select option
          const opt = document.createElement('option');
          opt.value = intent.id;
          opt.textContent = intent.tag;
          select.appendChild(opt);
        });
      });
}

function addIntent() {
  const tag = document.getElementById('intent_tag').value.trim();
  const response = document.getElementById('intent_response').value.trim();
  if (!tag || !response) return alert("Fill both");
  fetch('/admin/intents', {
    method: 'POST',
    headers: {'Content-Type': 'application/json'},
    body: JSON.stringify({tag, response})
  }).then(() => {
    loadIntents();
    document.getElementById('intent_tag').value = '';
    document.getElementById('intent_response').value = '';
```

```
    });
}

function updateIntent(id) {
    const tag = document.getElementById(`tag-${id}`).textContent.trim();
    const response = document.getElementById(`resp-${id}`).textContent.trim();
    fetch(`/admin/intents/${id}`, {
        method: 'PUT',
        headers: {'Content-Type': 'application/json'},
        body: JSON.stringify({tag, response})
    }).then(() => alert("Updated!"));
}

function deleteIntent(id) {
    if (!confirm("Delete intent? Patterns bhi delete honge")) return;
    fetch(`/admin/intents/${id}`, {method: 'DELETE'}).then(() => loadIntents());
}

function addPattern() {
    const intent_id = document.getElementById('intent_select').value;
    const pattern = document.getElementById('new_pattern').value.trim();
    if (!intent_id || !pattern) return alert("Select intent & type pattern");
    fetch('/admin/patterns', {
        method: 'POST',
        headers: {'Content-Type': 'application/json'},
        body: JSON.stringify({intent_id, pattern})
    }).then(() => {
        document.getElementById('new_pattern').value = '';
```

```
        alert("Pattern added!");
    });
}

window.onload = loadIntents;
</script>
```

Note: Edit for patterns abhi simple nahi, baad mein add kar sakte ho. Abhi add/delete intent + add pattern.

Run & test → /admin pe jaao, intents add karo, patterns daalo.

Next Phase Checklist:

☑ Admin mein new intent add?

☑ Pattern add hone ke baad chat mein test?

Phase 6: Testing, Polish & Final Recap

Goal: Sab test karo.

Testing Checklist:

☑ "hi bro" → greeting

☑ "fees batao please" → fees (lemmatize + TF-IDF handle karega)

☑ "kaisa chal raha hai?" → how_are_you

☑ "kuch samajh nahi aaya" → fallback

☑ Admin se new intent/pattern add karo → reload → chat mein work kare

☑ Logs DB mein sahi?

☑ Threshold adjust (0.3-0.6) test karo better matching ke liye

Polish:

- Chat UI mein confidence dikhao optional (response mein add {"response": "...", "confidence": max_sim})

- More patterns add karo synonyms ke liye (e.g. "cost", "price" for fees)

- Stopwords remove kar sakte ho clean_text mein (from nltk.corpus import stopwords)

Final Project Recap (Part 1 + 2 Complete!):

- Rule-based se AI-based upgrade

- Features: NLP intent detection (TF-IDF + cosine), multiple patterns per intent, fallback, admin manage, logs

- 24/7, accurate, scalable

- Matches project synopsis: NLP techniques, intent classification, predefined data, web integration

How to deploy locally: Same as Part 1 (python app.py)

For future (if want more):

- Add random response per intent (list of responses)

- Use deep learning (PyTorch LSTM for better accuracy)

- Voice (speech_recognition lib)

- Multilingual (googletrans)

Bhai, Part 2 done! 🚀

Achhe se test kar, koi issue ho to screenshot bhej ya error bata.

Next step: project report ya presentation? Ya improvements? Bol!